

天津大学

《智能搜索与问答》实验报告



Lab2 word2vec 实现及实验

学 号 3021244141
姓 名 于梦洁
学 院 智能与计算学部
专 业 计算机科学与技术
年 级 2021 级
班 级 2 班

2024 年 3 月 30 日

目录

一、 数据准备与处理	3
二、 模型与算法介绍	4
1. 任务定义	4
2. Word2vec 模型介绍	4
2.1 SkipGram	5
3. Word2vec 模型训练	6
3.1 构建训练数据	6
3.2 计算 loss	6
3.3 保存模型	6
4. Word2vec 模型测试	7
4.1 加载训练好的词向量	7
4.2 输入“***”	7
4.3 输出	7
三、 实验流程	7
四、 实验结果	10
五、 结果分析	12
六、 心得体会	12

一、 数据准备与处理

在本实验中，使用的数据集是“dataset/news.txt”。

体育 马晓旭意外受伤让国奥警惕 无奈大雨格外青睐国奥队 记者傅亚雨沈阳报道 来到沈阳，国奥队依然没有摆脱雨水的困扰。7月31日下午6点，国奥队的日常训练再度受到大雨的干扰，无奈之下队员们只慢跑25分钟就草草收场。31日上午10点，国奥队在奥体中心外场训练的时候，天就是阴沉沉的，气象预报显示当天下午沈阳就有大雨，但幸好队伍上午的训练没有受到任何干扰。下午6点，当球队抵达训练场时，大雨已经下了几个小时，而且丝毫没有停下来的意思。抱着试一试的态度，球队开始了当天下午的例行训练，25分钟过去了，天气没有任何转好的迹象，为了保护球员们，国奥队决定中止当天的训练。全队立即返回酒店。在雨中训练对足球队来说并不是什么稀罕事，但在奥运会即将开始之前，全队变得“娇贵”了。在沈阳最后一周的训练，国奥队首先要保证现有的球员不再出现意外的伤病情况以免影响正式比赛，因此这一阶段控制训练受伤、控制感冒等疾病的出现被队伍放在了相当重要的位置。而抵达沈阳之后，中后卫冯潇霆就一直没有训练，冯潇霆是7月27日在长春患上了感冒，因此也没有参加29日跟塞尔维亚的热身赛。队伍介绍说，冯潇霆并没有出现发烧症状，但为了安全起见，这两天还是让他静养休息，等感冒彻底好了之后再恢复训练。由于有了冯潇霆这个例子，因此国奥队对雨中训练就显得特别谨慎，主要是担心球员们受凉而引发感冒，造成非战斗减员。而女足队员马晓旭在热身赛中受伤导致无缘奥运的前科，也让在沈阳的国奥队现在格外警惕，“训练中不断嘱咐队员们要注意动作，我们可不能再出这样的事情了。”一位工作人员表示。从长春到沈阳，雨水一路伴随着国奥队，“也邪了，我们走到哪儿雨就下到哪儿，在长春几次训练都被大雨给搅和了，没想到来沈阳又碰到这种事情。”一位国奥球员也对雨水的“青睐”有些不解。

图 1. 数据集示例

数据预处理过程在文件“dataprocess.py”中实现，使用“class DataPreprocess()”。在这个过程中一共分为 3 步骤，分别为加载数据、过滤文本和 jieba 分词以及去停用词。

```
class DataPreprocess():
    def __init__(self, inputdata_path, stopword_path):
        self.dataad, self.labeled = self.dataload(inputdata_path, stopword_path)

    1 个用法
    def dataload(self, inputdata_path, stopword_path):
        train_data = pd.read_csv(inputdata_path, delimiter='\\t',
                                  lineterminator='\\n', encoding='UTF-8', header=None)
```

图 2. 加载数据

```
def wordcut(self, sentence):
    criterion = r'[\u4e00-\u9fa5a-zA-Z]+' # 文本正则化，只留下汉字和英文
    sentence = re.findall(criterion, sentence)
    sentence = "".join(sentence) # 拼接列表为字符串
    sentence = jieba.lcut(sentence) # jieba分词
    sentence = self.stopworddelete(sentence) # 去停止词
    return sentence
```

图 3. 过滤文本和 jieba 分词

```
def stopworddelete(self, sentence):
    word_num = 0
    while(word_num < len(sentence)):
        if sentence[word_num] in self.stopword:
            sentence.remove(sentence[word_num])
            word_num -= 1
        word_num += 1
    return sentence
```

图 4. 去停用词

使用命令“python dataprocess.py”对数据进行预处理得到分词后的文件 cutdata.txt。

```
马晓旭 意外 受伤 国奥 警惕 无奈 大雨 格外 青睐 殷家 军 记者 傅亚雨 沈阳 报道 来到 沈阳 国奥队 依然 没有 摆脱 雨水 困扰 月 日 下午 点 国奥队 日常 训练 再度  
大雨 干扰 无奈 之下 队员 慢跑 分钟 草草收场 日 上午 点 国奥队 奥体中心 外场 训练 天 阴沉沉 气象预报 显示 当天 下午 沈阳 大雨 幸好 队伍 上午 训练 没有 干扰  
下午 点 当 球队 抵达 训练场 时 大雨 已经 几个 小时 丝毫 没有 停下来 意思 抱 试一试 态度 球队 当天 下午 例行 训练 分钟 过去 天气 没有 转好 迹象 保护 球员  
国奥队 决定 中止 当天 训练 全队 立即 返回 酒店 雨 中 训练 足球队 稀罕 事 奥运会 即将 之前 全队 变得 娇贵 沈阳 最后 一周 训练 国奥队 保证 现有 球员 不再 出  
现意外 伤病 情况 影响 正式 比赛 这一 阶段 控制 训练 受伤 控制 感冒 疾病 出现 队伍 放在 相当 重要 位置 抵达 沈阳 之后 中 后卫 冯潇霆 一直 没有 训练 冯潇霆  
月 日 长春 患上 感冒 没有 参加 日 塞尔维亚 热身赛 队伍 介绍 说 冯潇霆 没有 出现 发烧 症状 安全 两天 静养 休息 感冒 彻底 之后 恢复 训练 冯潇霆 例子 国奥队  
对雨中 训练 显得 特别 谨慎 主要 担心 球员 受凉 引发 感冒 造成 非战斗 减员 女足 队员 马晓旭 热身赛 中 受伤 导致 无缘 奥运 前科 沈阳 国奥队 现在 格外 警惕  
训练中 不断 嘱咐 队员 注意 动作 不能 再出 事情 一位 工作人员 表示 长春 沈阳 雨水 一路 伴随 国奥队 邪 走 雨 长春 几次 训练 大雨 搅和 没想到 沈阳 碰到 这种  
事情 一位 国奥 球员 雨水 青睐 不解
```

图 5. 数据集预处理得到分词的示例

二、 模型与算法介绍

1. 任务定义

在 NLP 中，要让计算机读懂文本语言，首先要对文本进行编码。常见的编码如独热编码（one-hot encoding），词袋模型（BOW，bag of words），词向量模型（word embedding）。

本实验实现的 word2vec 就是词向量模型中的一种，它是 google 在 2013 年发布的工具。它通过将所有的词向量化，来定量的度量词与词之间的关系，挖掘词之间的联系。

2. Word2vec 模型介绍

Word2vec 是一种用于生成词向量的模型，它能够将词语映射到一个连续的向量空间中，使得语义相近的词语在向量空间中的距离也相近。词向量是自然语言处理中的一种重要技术，它能够捕捉词语之间的语义和语法关系，为文本分析、情感分析、文本分类等任务提供有力支持。

word2vec 工具主要包含两个模型：连续词袋模型（CBOW，continuous bag of words）和跳字模型（skip-gram）。在本实验中使用的模型为 SkipGram 模型。

```

class SkipGramModel(nn.Module):
    def __init__(self, emb_length, emb_dimension):
        super(SkipGramModel, self).__init__()
        self.emb_length = emb_length
        self.emb_dimension = emb_dimension
        self.u_embedding = nn.Embedding(self.emb_length, self.emb_dimension, sparse=True)
        self.v_embedding = nn.Embedding(self.emb_length, self.emb_dimension, sparse=True)
        initrange = 0.5/self.emb_dimension
        self.u_embedding.weight.data.uniform_(-initrange, initrange)
        self.v_embedding.weight.data.uniform_(-0, 0)

    1 个用法
    def forward(self, pos_u, pos_v, neg_v):
        pos_embu = self.u_embedding(pos_u)
        pos_embv = self.v_embedding(pos_v)
        neg_embv = self.v_embedding(neg_v)
        # 逐元素相乘计算正样本对相似度, log以e为底
        pos_score = torch.mul(pos_embu, pos_embv)
        pos_score = F.logsigmoid(torch.sum(pos_score, dim=1).squeeze())
        # 计算负样本相似度, neg_embv.shape=[batch_size, neg_count, emb_dimension]
        neg_score = torch.matmul(neg_embv, pos_embu.unsqueeze(2)).squeeze()
        neg_score = F.logsigmoid(-1*neg_score)
        return -1*(torch.sum(neg_score)+torch.sum(pos_score))

```

图 6. SkipGram 模型代码实现

2.1 SkipGram

Skip-gram 模型从 target word 对 context 的预测中学习得到 word vector, 该名称源于该模型在训练时会上下文环境里的 word 进行采样。Skip-gram 是根据目标词去预测周围词来训练得到词向量, 如图 7 是根据 $W(t)$ 去预测 $W(t-2), W(t-1), W(t+1), W(t+2)$ 。

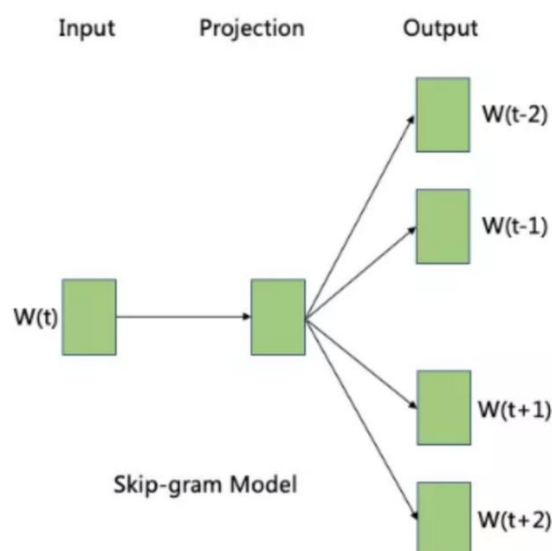


图 7. SkipGram 模型

3. Word2vec 模型训练

Word2vec 模型训练分为 3 步骤，分别为构建训练数据、计算 loss 以及保存模型。

3.1 构建训练数据

```
pos_pairs = self.data.getpospair(self.batch_size,
                                self.window_size)
neg_v = self.data.getnegv(pos_pairs, self.neg_count)
pos_u = [pair[0] for pair in pos_pairs] # 取出中心词
pos_v = [pair[1] for pair in pos_pairs] # 取出背景词
```

图 8. 构建训练数据

3.2 计算 loss

计算 loss 值中，使用了 SGD 优化器，其中学习率设置的值为 0.005。

```
self.optimizer.zero_grad()
loss = self.skip_gram_model.forward(pos_u, pos_v, neg_v)
loss.backward()
self.optimizer.step()
```

图 9. 计算 loss

3.3 保存模型

```
self.skip_gram_model.savemodel(
    self.data.id_word, self.output_file_name)

def savemodel(self, id_word, outdata_path):
    if torch.cuda.is_available:
        # numpy无法直接从CUDA中读取数据，需要先将CUDA中的数据转移到cpu中
        embedding = self.u_embedding.weight.data.cpu().numpy()
    else:
        embedding = self.u_embedding.weight.data.numpy()
    with open(outdata_path, 'w', encoding='UTF-8') as f:
        f.write('词语数量: {} 嵌入维度: {}\n'.format(len(id_word), self.emb_dimension))
        for i, word in id_word.items():
            e = embedding[i]
            e = ' '.join(map(lambda x: str(x), e))
            f.write('{} {}\n'.format(word, e))
```

图 10. 保存模型

4. Word2vec 模型测试

Word2vec 模型测试包括 3 个步骤，分别为加载训练好的词向量、输入 “***” 以及输出相似度最高的 20 个词。

4.1 加载训练好的词向量

```
f=open('datasave/word_embedding_pretrained.txt',encoding='UTF-8')
f.readline()
all_embeddings=[]
all_words=[]
word2id=dict()
```

图 11. 加载训练好的词向量

4.2 输入 “***”

当终端出现 “Word: ” 时即可输入想要查询的词汇了。

```
23 word=input('Word: ')
```

图 12. 输入 “***”

4.3 输出

```
d = cosine_similarity(embedding, all_embeddings)[0]
d=zip(all_words, d)
d=sorted(d, key=lambda x:x[1], reverse=True)
for w in d[:20]:
    if len(w[0])<2:
        continue
    print(w)
```

图 13. 输出相似度高的 20 个词

三、 实验流程

1. 使用 conda 创建虚拟环境

点击 Anaconda Powershell Prompt，打开 powershell。

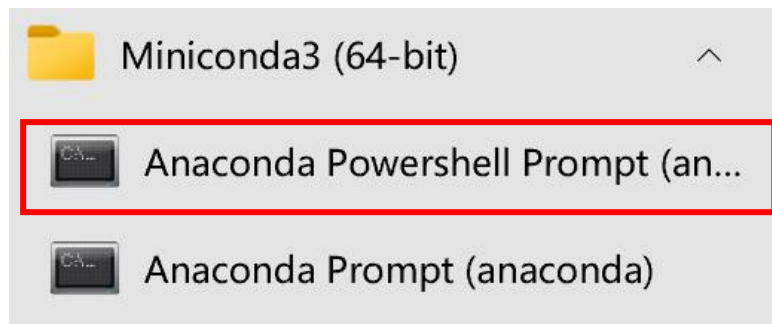


图 14. powershell 打开入口

使用命令 “conda create -n lab2 python=3.8” 创建需要的环境，然后使用命令 “conda activate lab2” 激活环境，之后进入代码所在文件夹。

```

Anaconda Powershell Prompt (anaconda)
(base) PS C:\Users\admin> conda create -n lab2 python=3.8
Retrieving notices: ...working... done
Channels:
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

environment location: D:\software\anaconda\envs\lab2

added / updated specs:
- python=3.8

The following packages will be downloaded:



| package       | build      | size   |
|---------------|------------|--------|
| libffi-3.4.4  | hd77b12b_1 | 122 KB |
| sqlite-3.45.3 | h2bbff1b_0 | 973 KB |
| Total:        |            | 1.1 MB |



The following NEW packages will be INSTALLED:

ca-certificates pkgs/main/win-64::ca-certificates-2024.3.11-haa95532_0
libffi pkgs/main/win-64::libffi-3.4.4-hd77b12b_1
openssl pkgs/main/win-64::openssl-3.0.13-h2bbff1b_0
pip pkgs/main/win-64::pip-23.3.1-py38haa95532_0
python pkgs/main/win-64::python-3.8.19-h1aa4202_0
setuptools pkgs/main/win-64::setuptools-68.2.2-py38haa95532_0
sqlite pkgs/main/win-64::sqlite-3.45.3-h2bbff1b_0
vc pkgs/main/win-64::vc-14.2-h21ff451_1
vs2015_runtime pkgs/main/win-64::vs2015_runtime-14.27.29016-h5e58377_2
wheel pkgs/main/win-64::wheel-0.41.2-py38haa95532_0

Proceed ([y]/n)? y

Downloading and Extracting Packages:
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#   $ conda activate lab2
#
# To deactivate an active environment, use
#
#   $ conda deactivate
#
(base) PS C:\Users\admin> conda activate lab2
(lab2) PS C:\Users\admin> cd D:\DDD\大三下\智能搜索与问答\lab2\word2vec实现
(lab2) PS D:\DDD\大三下\智能搜索与问答\lab2\word2vec实现>

```

图 15. Lab2 环境配置

2. 运行实验

输入命令“pip install scikit-learn”，安装实验运行所需要的库“scikit-learn”。

```
(lab2) PS D:\DDD\大三下\智能搜索与问答\lab2\word2vec实现> pip install scikit-learn
Collecting scikit-learn
  Downloading scikit_learn-1.3.2-cp38-cp38-win_amd64.whl.metadata (11 kB)
Collecting numpy<2.0,>=1.17.3 (from scikit-learn)
  Downloading numpy-1.24.4-cp38-cp38-win_amd64.whl.metadata (5.6 kB)
Collecting scipy>=1.5.0 (from scikit-learn)
  Downloading scipy-1.10.1-cp38-cp38-win_amd64.whl.metadata (58 kB)
    59.0/59.0 kB 518.5 kB/s eta 0:00:00
Collecting joblib>=1.1.1 (from scikit-learn)
  Downloading joblib-1.4.0-py3-none-any.whl.metadata (5.4 kB)
Collecting threadpoolctl>=2.0.0 (from scikit-learn)
  Downloading threadpoolctl-3.5.0-py3-none-any.whl.metadata (13 kB)
Downloading scikit_learn-1.3.2-cp38-cp38-win_amd64.whl (9.3 MB)
    9.3/9.3 MB 14.1 MB/s eta 0:00:00
Downloading joblib-1.4.0-py3-none-any.whl (301 kB)
    301.2/301.2 kB ? eta 0:00:00
Downloading numpy-1.24.4-cp38-cp38-win_amd64.whl (14.9 MB)
    14.9/14.9 MB 21.9 MB/s eta 0:00:00
Downloading scipy-1.10.1-cp38-cp38-win_amd64.whl (42.2 MB)
    42.2/42.2 MB 21.8 MB/s eta 0:00:00
Downloading threadpoolctl-3.5.0-py3-none-any.whl (18 kB)
Installing collected packages: threadpoolctl, numpy, joblib, scipy, scikit-learn
Successfully installed joblib-1.4.0 numpy-1.24.4 scikit-learn-1.3.2 scipy-1.10.1 threadpoolctl-3.5.0
```

图 16. 安装库“scikit-learn”

运行命令行“python dataprocess.py”对数据集'dataset/news.txt'，进行数据预处理得到分词后的文件 cutdata.txt，再利用 word2vec 模型进行词嵌入得到 word_embedding.txt 文件，最后测试效果。

由于 datasave 文件夹已经包含经处理过后的 cutdata_prepare.txt 和 word_embedding_pretrained.txt，因此可以直接用来测试。只需修改文件 test.py 中读取的文件。

```
5 """
6 from sklearn.metrics.pairwise import cosine_similarity
7 import numpy
8 f=open('datasave/word_embedding_pretrained.txt',encoding='UTF-8')
9 f.readline()
```

图 17. 修改文件使用经处理的 word_embedding_pretrained.txt

输入命令“python test.py”进行测试，分别输入 word “男人、女人、女孩、女孩子、女生、男孩、但是”，得到相应的测试结果。

四、 实验结果

```
(lab2) PS D:\DDD\大三下\智能搜索与问答\lab2\word2vec实现> python test.py
Word: 男人
('男人', 0.9999999999999998)
('女人', 0.9850906586593999)
('女孩', 0.9609120069650579)
('漂亮', 0.9462240616256592)
('从小', 0.9410057989322115)
('女孩子', 0.9281780932955783)
('老人', 0.9268355951186703)
('女生', 0.9266408686379426)
('聪明', 0.9254501398910284)
('心中', 0.920820431395281)
('上面', 0.9206951714624667)
('小孩', 0.9184986518300134)
('厨房', 0.9180854913045617)
('伟大', 0.9166695142203515)
('性格', 0.9157306846170286)
('看似', 0.9155756914142947)
('塑造', 0.9148699599061163)
('主人', 0.9131037901679326)
('一生', 0.9128269059932581)
('味道', 0.9113431542571213)
Word: 女人
('女人', 0.9999999999999998)
('男人', 0.9850906586593999)
('女孩', 0.9559860282022643)
('漂亮', 0.9369953074814886)
('从小', 0.9314061533747022)
('可爱', 0.9251287773795802)
('老人', 0.9200216307181441)
('美丽', 0.9191956428345018)
('女孩子', 0.917869958981975)
('女生', 0.9177506820487591)
('一条', 0.9150402732598517)
('装扮', 0.9130777494027047)
('时髦', 0.9107614470181721)
('味道', 0.9096288481980568)
('眼神', 0.9092421288548785)
('一只', 0.9078984121695861)
('厨房', 0.9066685221731043)
('MM', 0.9065768316662512)
('主人', 0.9057264253003514)
('男生', 0.9056369645882523)
Word: 女孩
('女孩', 1.0)
('老人', 0.9684833513978921)
('从小', 0.9669913868382188)
('女生', 0.9626952079614791)
('男人', 0.9609120069650579)
('女人', 0.9559860282022643)
('男生', 0.9553895468096723)
('女孩子', 0.9536060470633902)
('男孩', 0.9501035279337028)
('一群', 0.9499322835376331)
('小孩', 0.9497789451345049)
('长大', 0.9493060858795148)
('前面', 0.9460555488183939)
('小时候', 0.9453957239102793)
('主人', 0.9424557858404606)
('外面', 0.9420695207022287)
('爸爸', 0.9419027366786338)
('送给', 0.9416400394899566)
('旁边', 0.9415767939364466)
('眼里', 0.9405267348740721)
('眼神', 0.9388809308124304)
```

```

Word: 女孩子
('女孩子', 1.0000000000000002)
('男生', 0.9903964521080107)
('女生', 0.9896971668436465)
('小时候', 0.9891644093100337)
('好玩', 0.9884567656228053)
('我演', 0.9877397828501784)
('平常', 0.987006157547461)
('小孩子', 0.9853793772218966)
('MM', 0.9852557522244653)
('从小', 0.9848596712429006)
('长大', 0.9846282608340758)
('一辈子', 0.9843644825466141)
('做事', 0.9843127194038742)
('安静', 0.9842968608866884)
('外面', 0.9840831504173834)
('厉害', 0.9839253678049393)
('年纪', 0.9837205394451626)
('严肃', 0.9833551710464035)
('聪明', 0.9832701563994553)
Word: 女生
('女生', 1.0000000000000002)
('女孩子', 0.9896971668436465)
('男生', 0.9852381847057892)
('老人', 0.984154393355812)
('从小', 0.9821191504577105)
('小时候', 0.9802514505498454)
('主人', 0.9775533849644824)
('MM', 0.9772411137058623)
('好玩', 0.9768873146342514)
('外面', 0.976273269964415)
('聪明', 0.9748511624475793)
('小孩', 0.9744080977898077)
('周围', 0.974291472555698)
('长大', 0.9726687616073091)
('平常', 0.9721666443329793)
('我演', 0.9720583390654033)
('严肃', 0.9719401990475108)
('眼神', 0.9718333781807648)
('安静', 0.9716183438254641)
('老外', 0.9705568041315834)
('小孩子', 0.9705338019545222)
('样子', 0.9702871291636048)
Word: 男孩
('男孩', 1.0)
('姑娘', 0.9914234016945594)
('童年', 0.9890772374242394)
('出身', 0.9888490890419107)
('多岁', 0.9884219742918776)
('眼里', 0.988221115905447)
('平凡', 0.9880868426210442)
('商人', 0.9880223278926474)
('一群', 0.9871768338802713)
('总有', 0.986800330460682)
('幻想', 0.9854793086372271)
('送给', 0.9854347333489091)
('含义', 0.9848435301901081)
('活着', 0.98483441147662)
('那段', 0.9848173570123178)
('英国人', 0.9847113860555785)
('比喻', 0.9846460576444254)
('小孩子', 0.9845652071187871)
('举个', 0.9845069383063338)
('成名', 0.9841311391307532)
('角落', 0.9841093443914388)
('有点像', 0.9840988230186088)
('圈子', 0.984010318856414)
Word: 但是
Cannot find this word

```

图 18. 测试结果

五、 结果分析

在测试的过程中，我一共使用 7 个词，其中对于“男人”、“女人”、“女孩”、“女孩子”、“女生”和“男生”，分别给出了相似度最高的 20 个词，以及每个词对应可能的概率。其中，相似度评分越接近 1 表示越相似，越接近 0 表示越不相似。对于“但是”这个词，由于它不是训练过的词语 embedding，因此输出“Cannot find this word”。

从结果中可以发现，对于每一个词语都与自身的匹配得分为 1 或者无限接近于 1，这是因为输入的词语是模型的关键词。

可以发现，对于“男人”和“女人”这两个词，它们之间的相似度得分很高，表明模型认为它们之间有很强的关联性。对于“男人”和“女人”、“女孩”、“从小”等词汇高度相关，这可能是因为大多数情况下，人们会将男性或女性的概念与年龄（从小）、性别（女孩/男孩）以及社会角色（女人/男人）联系起来，说明这些词语与性别、年龄等因素有关。“女孩”与“女生”、“男孩子”、“从小”等词汇高度相关，这可能是因为这些词汇通常用来描述年轻的女性或女性儿童，并且与她们的成长和发展有关。

六、 心得体会

通过这次实验，我更加深入地了解自然语言处理中的 Word2vec 算法，包括其原理、优缺点以及应用场景。掌握 Word2vec 算法对于自然语言处理领域是非常重要的，它为我们提供了一种有效的方式来处理文本数据并提取其中的语义信息。此外，我还学习了如何使用 Python 和 scikit-learn 库实现 Word2vec 模型的训练和评估。

通过实践和总结经验，我不仅加深了对 Word2vec 原理和应用的理解，还强调了文本预处理和参数调优的重要性，这对于提高模型性能至关重要。

Word2vec 是一种基于神经网络的词向量表示方法，它可以将词汇映射到高维空间中的向量，使得语义相近的词语具有相似的向量表示。这有助于解决传统词袋模型无法捕捉词汇之间关系的局限性。

值得一提的是，Word2vec 可以应用于很多自然语言处理任务，如文本分类、情感分析、机器翻译等。通过 Word2vec，我们可以将文本数据转化为数值型特征，从而更好地利用机器学习算法进行分析。

在进行实验前，对数据进行预处理十分重要。在进行词向量训练之前，需要对原始文本进行预处理，包括分词、去除停用词、标点符号等。这些操作可以提高模型的学习效率，减少噪声的影响。

总之，通过这次实验，我对 Word2vec 有了更深入的理解，也提高了自己的编程能力和数据分析能力。在未来的学习和工作中，我将继续努力，提高自己的技能水平。